TIP For your convenience, I prepared a script that performs all the previous operations with a single command. You can run it to create a local Kubernetes cluster with minikube, enable the Ingress NGINX add-on, and deploy the backing services used by Polar Bookshop. You'll find the create-cluster.sh and destroy-cluster.sh files in the kubernetes/platform/development folder that you have just copied over to your `polar-deployment` repository. On macOS and Linux, you might need to make the scripts executable via the `chmod +x create-cluster.sh` command.

Let's conclude this section by packaging Edge Service as a container image and loading the artifact to the local Kubernetes cluster. Open a Terminal window, navigate to the Edge Service root folder (edge-service), and run the following commands:

```
$ ./gradlew bootBuildImage
$ minikube image load edge-service --profile polar
```

In the next section, you'll define an Ingress object and configure it to manage external access to the Polar Bookshop system running in a Kubernetes cluster.

9.5.2 Working with Ingress objects

Edge Service takes care of application routing, but it should not be concerned with the underlying infrastructure and network configuration. Using an Ingress resource, we can decouple the two responsibilities. Developers would maintain Edge Service, while the platform team would manage the ingress controller and the network configuration (perhaps relying on a service mesh like Linkerd or Istio). Figure 9.6 shows the deployment architecture of Polar Bookshop after introducing an Ingress.